

Bank Settlement Reconciliation — May & June 2026

Summary write-up. Every number below comes out of `Bancapp_Reconciliation.ipynb`; run it top to bottom and it reproduces.

1. Monthly reconciliation results

	May 2026	June 2026
Transactions to settle	328 · ₹1,32,13,754	318 · ₹1,28,29,424 (incl. 30 May backlog)
Bank credits received	252 · ₹1,21,20,212	250 · ₹1,14,20,470
MATCHED	226 groups · ₹1,15,73,495	228 groups · ₹1,09,40,058
PARTIAL_MATCH	3 groups · ₹74,257	2 groups · ₹36,850
OPEN	30 groups · ₹15,66,003	32 groups · ₹18,52,516
EXCEPTION	15 groups · ₹6,31,241	12 groups · ₹5,41,884
Cardinality	1:1 = 209 · 1:N = 8 · N:1 = 11 · N:M = 1	1:1 = 214 · 1:N = 6 · N:1 = 9 · N:M = 1

87.6% of May's transaction value and 85.3% of June's is fully matched at zero variance. Across everything that did match, the total unexplained difference is **₹941 in May and ₹667 in June** — small enough to be bank fees, and every rupee of it is named in the exception report.

2. Matching methodology and pass order

The bank writes our reference inside its narration, but rarely cleanly. Four real cases from the data: `SETTLE/PRMA Y95565484/PAYOUT` (a space dropped into the middle), `SETTLE/prmay85386435/PAYOUT` (lower case), `SETTLE/NEFTPRMAY58438711/PAYOUT` (channel name glued on the front), `SETTLE/PRMAY832847/PAYOUT` (chopped short). So the first thing the code does is tidy the narration — upper case it and delete every space, which fixes the first three. For the fourth we look for ledger references that *start with* what the bank wrote; if exactly one does, that is the answer, and if more than one does we leave it alone rather than guess.

Six passes then run in a fixed order, most specific first:

- Sale-and-reversal pairs inside our own books** — a sale and its reversal on the same reference cancel out, so no credit is ever coming. Removed first so they never sit in the report looking unpaid.
- Batch payments, N:1** — `SETTLEMENT BATCHMAY006` pays a whole batch at once. Where the narration says `NET OF CHGS 330.26` the bank kept a fee, and we add it back before comparing, turning an apparent shortfall into an exact tie.
- Instalments, 1:N** — `PART SETTLEMENT PRMAY48951595-P1/-P2/-P3`. All instalments are gathered *before* comparing, so no single leg can claim to have paid the whole transaction.
- Reference match, 1:1** — the main pass, about 85% of volume.
- Merchant-day sweeps, N:M** — `NET SETTLEMENT M1016 2026-05-19`, where the bank netted a whole day for one merchant.
- Whatever is left** — transactions with no credit become OPEN, credits with no transaction become orphan credits.

Why the sweep runs last. A merchant-day sweep grabs *everything* for that merchant on that date. Run early, it swallows transactions that have their own individual credit — `TXNMAY00104` falls inside the `M1010` window on 19 May but is genuinely paid on its own line `BLMAY00104`. Running the sweep only after reference matching means it can pick up nothing but the true leftovers. This is the single most important ordering decision in the engine.

3. How duplicate / redundant matching was prevented

The code keeps two lists — transactions already used, and bank credits already used. Every match in every pass has to go through one function, `make_group()`. Before it creates anything it checks whether any record involved is already on those lists; if even one is, the whole match is refused and nothing changes. There is no way around this, because there is no other way to create a match.

In practice: the bank paid `PRMAY20770971` twice, ₹12,850.78 on two different UTRs. The first credit settles the transaction; the second is refused and booked as a duplicate credit instead of quietly doubling our income. **11 credits worth ₹4,96,223** were caught this way. The notebook ends by asserting that each of the 1,148 records sits in exactly one group — if that were untrue it stops with an error rather than printing wrong numbers.

4. May backlog carried into June

June is **not** reconciled on its own: the 30 items open at 31 May are added to the June pool, so a May sale can be cleared by a June credit without being counted twice anywhere.

	Count	Value
Open at 31 May (carried forward)	30	₹15,66,003
Cleared during June	22	₹11,88,928
Still open at 30 June	8	₹3,77,074

All 22 arrived as `SETTLEMENT PRMAY59755172 PREV CYCLE` and tied to the exact rupee. An `origin` column on every output row says whether the transaction came from May or June, so the three views the brief asks for — June settled in June, May cleared in June, still open — all read off the one file.

5. Lag vs exception rule

■ An item still open at month end is a *likely settlement lag* if it is 5 days old or less. Older than that, it is a *genuine exception*.

Five days covers a normal settlement cycle plus a weekend, so the money is expected in the next run. The data supports it almost perfectly: of May's 30 open items, the 22 aged 5 days or less at 31 May **all** settled in June, and the 8 older ones **never** did. At 30 June the split is 18 lag and 14 genuine exceptions.

6. Exception summary (both months)

Category	Items	Value	Recommended action
Unsettled transaction	22	₹11,78,729	Chase the acquirer — past the normal window
Duplicate credit	11	₹4,96,223	Confirm with the bank, reverse the extra credit
Orphan credit	9	₹4,22,677	Trace the UTR, identify who sent it
Internal self-netting pair	7	₹2,54,224	No action — cancels out, no credit is due
Short settlement	5	₹1,11,106	Recover the shortfall — undeclared fee

Exceptions are counted per month-end, so the 8 May items still unsettled at 30 June appear under both months; the June rows are the closing position. Ageing is reported in 0-7d / 8-15d / 16-30d / 31-60d / 60d+ buckets in `exception_report.csv` and on the dashboard.

Short settlements matter more than their size suggests: five groups worth ₹1,11,106 came up short by a combined ₹1,608, under 1% each — the fingerprint of a fee deducted without being declared. We deliberately do *not* absorb these into MATCHED. Money is missing, so they stay PARTIAL_MATCH and stay visible.

Input validation: 46 checks across the four files — **0 errors, 14 warnings**. Every warning is a payment reference used by two transactions, which turned out to be the expected sale-and-reversal pairs. Nothing was silently dropped; all of it is in `input_validation_report.csv`.

7. Output format design (Task 4)

The main file is **one row per match group**, listing every transaction id and bank line id in that group alongside the group's own totals and variance — a parent-child structure flattened into one table.

Shape	Best for operations	Best for audit
1:1	One row, transaction and credit side by side. Nothing else needed.	Same row, plus <code>match_method</code> showing <i>how</i> the reference resolved.
1:N	The transaction is the unit of work; ops only care whether the instalments add up.	Every instalment with its sequence (P1/P2/P3), so the build-up is re-performable.
N:1	The batch is the unit of work — show batch total and variance, open members only on a break.	Every member plus the added-back charge, so the net credit derives from the ledger.
N:M	One line: "merchant M1016, 19 May, nets to ₹49,631".	Every sale, refund and credit leg — the netting itself is the control being tested.

Flat table vs parent-child. A flat table is easy to filter and pivot, but if group totals repeat on every child row a careless `SUM(variance)` double-counts. A strict parent-child design (separate group and member files) never double-counts and models 1:N and N:M honestly, but forces a join for every question. This output takes the middle path: one row per group keeps the file safe to sum and easy for ops to filter, the id lists give the auditor a full trace back to individual records, and `summary_control_report.csv` carries pre-totalled numbers so nothing is added up by hand.

8. Assumptions

A difference of ₹1 or less counts as matched (rounding, not a break). Bank charges are added back **only when the narration declares them** — undeclared shortfalls are reported, never absorbed. Month end means calendar month end. Every ledger row is SUCCESS, so all are expected to settle; non-successful rows would be logged and excluded, and the check runs. The merchant code inside a plain UPI/SETTLE/ . . . narration is **not** used for matching — it disagrees with our own `merchant_id` in places, so only the reference is trusted.

9. Time spent

Roughly **3 hours 45 minutes**: 45 minutes reading the data and working out what the narration formats were hiding, 1 hour 45 minutes writing and testing the matching passes (the sweep-ordering bug took a good part of that), 40 minutes on the output files and dashboard, 35 minutes on this write-up.